

**SmartAssist: An Intelligent Email Triage and
Response Automation Platform**

PHASE I REPORT

Submitted by

RAGUL A	230701252
LOGAPRIYA S G	230701164
DIVYA K	230701082

in partial fulfillment for the award of the degree
of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI
ANNA UNIVERSITY: CHENNAI 600 025

September 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled “**SMARTASSIST: AN INTELLIGENT EMAIL TRIAGE AND RESPONSE AUTOMATION PLATFORM**” is the bonafide work of **RAGUL A (230701252), LOGAPRIYA S G (230701164), DIVYA K (230701082)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

“This project addresses the following Sustainable Development Goal(s):

SDG 9 - Industry, Innovation and Infrastructure: Uses AI-driven technologies to develop an innovative and efficient customer support infrastructure.

SDG 8 - Decent Work and Economic Growth: Automates repetitive support tasks and improves employee productivity through intelligent ticket management.

SIGNATURE

Dr. J. Manoranjini, Professor and Head,
Department of Computer Science and
Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

SIGNATURE

Ms. M. Bhavani Assistant Professor
Department of Computer Science and
Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

Customer support centers get an abundance of emails which need to be manually categorized, prioritized, routed, and assigned to support staff members. This process may take a lot of time and negatively affect the workloads and compliance with SLA. SmartAssist: Intelligent Email Triage and Response Automation Platform is proposed to automate the processes of managing the customers' support tickets.

The platform integrates different email services to receive support emails from customers. A specially trained multi-task DistilBERT model is used to classify the intent and priority of each received email. Depending on the predicted intent, the tickets are intelligently routed to a corresponding workspace or support mailbox. The workload-balanced assignment strategy assigns the tickets to the most suitable support agent depending on the current workload of the latter.

Besides, the solution uses Retrieval-Augmented Generation (RAG) to retrieve the SOPs, company policies, FAQs, and previous support emails. The document embeddings are saved in PostgreSQL using pgvector, and the information retrieved is given as additional context to the LLaMA model to generate the response draft. There is also a human-in-the-loop mechanism allowing the support staff to check and approve the response draft.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. Manoranjini, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Ms. M. Bhavani, M.E.**, Assistant Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project.

RAGUL A : 230701252

LOGAPRIYA S G : 230701164

DIVYA K : 230701082

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	iii
	LIST OF TABLES	viii
	LIST OF FIGURES	ix
	LIST OF ABBREVIATIONS	x
1.	INTRODUCTION	1
	1.1 GENERAL	1
	1.2 OBJECTIVE	2
	1.3 EXISTING SYTEM	3
	1.4 PROPOSED SYSTEM	5
2.	LITERATURE SURVEY	7
	2.1 LITERATURE REVIEW	7
	2.2 COMPARISION AND DISCUSSION	11
	2.3 CONCLUSION	12
3.	SYSTEM DESIGN	13
	3.1 SYSTEM ARCHITECTURE	13
	3.2 SYSTEM REQUIREMENTS	16
	3.2.1 SOFTWARE REQUIREMENTS	16
	3.2.2 HARDWARE REQUIREMENTS	18
	3.2.3 DATASET REQUIREMENTS	19
	3.2.4 DEPLOYMENT AND SCALING	20
	3.3 SYSTEM DESIGN	20
	3.3.1 ACTIVITY DIAGRAM	21

	3.3.2 DATAFLOW DIAGRAM	23
4.	PROJECT DESCRIPTION	25
	4.1 METHODOLOGIES	25
	4.2 MODULES	25
	4.2.1 DATASET COLLECTION AND PREPROCESSING	26
	4.2.2 MODEL TRAINING AND EVALUATION	27
	4.2.3 EMAIL INTEGRATION AND TICKET GENERATION	28
	4.2.4 DEPARTMENT ROUTING AND AGENT ASSIGNMENT	28
	4.2.5 KNOWLEDGE RETREIVAL AND RESPONSE GENERATION	29
	4.2.6 HUMAN-IN-THE-LOOP APPROVAL	30
5.	IMPLEMENTATION AND RESULT DISCUSSION	31
	5.1 IMPLEMENTATION AND RESULT	31
	5.1.1 USER INTERFACE AND EMAIL INTEGRATION	31
	5.1.2 MODEL PERFORMANCE EVALUATION	33
	5.1.3 ROUTING AND RESPONSE GENERATION	35
	5.2 DISCUSSION	36

6.	CONCLUSION	38
	6.1 CONCLUSION	38
	6.2 FUTURE WORK	39
	REFERENCES	40

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO.
3.1	Software Requirements of the Proposed System	17
3.2	Hardware Requirements of the Proposed System	19
4.1	All the 25 intents used for model training	26
4.2	Distribution of dataset across training and testing sets	27
5.1	Classification model evaluation	33

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
1.1	Existing Systems	4
1.2	Overview of the Proposed System Architecture	6
3.1	Architecture Diagram for the Entire System	14
3.2	Activity Diagram for the Active Environment	21
3.3	Activity Diagram for the Passive Environment	22
3.4	Data Flow Diagram o the system	24
4.1	Review and Response by Support Agent	30
5.1	SmartAssist Login Interface	32
5.2	SmartAssist Support Agent's Dashboard Interface	32
5.3	Confusion Matrix for the support intents	34
5.4	Training and Validation Loss across epochs	34
5.5	Ticket Review and Human in the loop Interface	36

LIST OF ABBREVIATIONS

Abbreviation	Full Form
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
SLA	Service Level Agreement
SOP	Standard Operating Procedure
NER	Named Entity Recognition
MTL	Multi-Task Learning
DPR	Dense Passage Retrieval
HITL	Human-in-the-Loop
TAT	Turnaround Time
SRS	Software Requirements Specification

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Customer Support has become an important part of modern business as people use technology-based means to convey their queries or complaints. Email is one of the most popular modes used to interact with support representatives. But organizations face numerous support emails each day, which make them difficult and time-consuming to process manually. The support representatives need to first go through the email to figure out what the customer requires, how urgent the query is, and then forward it to the concerned department.

Most traditional support systems utilizing emails as their communication channels are primarily based on manual processes when it comes to the process of ticket classification and management. This causes delays in responses, wrong routing of tickets, imbalance in the allocation of tickets to different support employees, and even difficulty in complying with Service Level Agreement (SLA). In light of the increase in number of requests from clients, doing all the tasks manually would be highly inefficient. Hence, the need for an automated system arises.

The recent advances in Artificial Intelligence and NLP have provided viable options for dealing with unstructured information in emails. AI models are capable of automatically determining the intent and priority of support queries, whereas intelligent routing can help redirect the ticket to the correct support space.

1.2 OBJECTIVE

The main aim of the SmartAssist: Intelligent Email Triage and Response Automation Platform is to automate the handling of customer support emails through Artificial Intelligence. The platform seeks to minimize the human work required in analyzing, interpreting, classifying, and re-routing customer support requests. Through automated analysis of the contents of incoming emails, SmartAssist determines the intention of the support request and predicts the priority, thus helping in managing tickets in an efficient manner.

Another goal of the project is to ensure that the support request is routed to the right workspace or department according to the predicted intention and capabilities of connected email accounts. Following the successful routing of the emails, the system allocates tickets to support agents through a load-balancing algorithm, favoring those agents who have a lower number of tickets.

This project seeks to facilitate intelligent and context-aware response generation through the use of a Retrieval-Augmented Generation (RAG) system architecture. The relevant data in the form of SOPs, SLA guidelines, FAQs, policy documents, and prior resolutions are retrieved through semantic search based on pgvector and then fed into a language model that leverages LLaMA for drafting responses. In general, SmartAssist seeks to improve response times, minimize human effort, and facilitate efficient ticket management

1.3 EXISTING SYSTEMS

There are numerous commercially available customer service systems that include Zendesk, ServiceNow, Salesforce Service Cloud, and Freshdesk. All of them offer support services to manage customer inquiries and tickets. They usually turn incoming requests received through different communication means like email, chat, web forms, and messaging apps into tickets/cases. Tickets/cases may be categorized, assigned to agents or teams, tracked, and monitored by the support staff.

Zendesk is a ticketing system that offers automation capabilities and support through AI technologies. It has an intelligent triage that allows classifying tickets based on such criteria as topics, sentiments, languages, as well as various routing that allows allocating tickets considering agents' availability, capacity, skills, priorities, and SLAs. Nonetheless, more sophisticated routing and AI features may require certain plans/configurations/channel requirements. There are also some limitations of certain routing features in regard to communication channels and features.

Another popular platform used for the enterprise is ServiceNow. It includes incident management, service request management, problem management, and process workflows via a ticket-based system. ServiceNow provides automation, workflow management, knowledge management, and AI capabilities for enterprise needs. Nevertheless, the implementation and customization of such an enterprise-level system might be rather resource-consuming, especially for smaller organizations, which need a specific support process.

In the same way, Salesforce Service Cloud that is highly integrated with Agentforce-based capabilities offers case management, omnichannel routing, knowledge integration, and AI-powered customer support. Cases can be classified and routed to relevant agents or queues, and AI-powered tools help with response generation and knowledge access. Nevertheless, all those capabilities are offered by the enterprise-level architecture and might be too complex for the needs of an organization, which only requires support automation via intelligent email management.

Additionally, there are platforms like Freshdesk and other helpdesk software solutions that also offer ticketing system, automation rules, agent assignments, and knowledge bases. Though these platforms help in improving the management of customer inquiries, companies may need to configure various rules, processes, categories, and routing conditions according to their needs.



Figure 1.1. Existing Systems

1.4 PROPOSED SYSTEM

The designed SmartAssist architecture is meant to be an intelligent email support automation system that would handle the entire process of customer support request life cycle. The architecture would incorporate email interaction, artificial intelligence, knowledge retrieval, intelligent routing, assignment of agents, and human supervision in one coherent process. The architecture is made up of several layers that work together to guarantee efficient processing and delivery of incoming support requests.

The first step as described in Figure 1.2 involves Email Integration Layer whereby customer requests are gathered from the mailboxes linked to the system using Gmail API, Microsoft Graph API or IMAP among others. Every email that comes in is normalized and then added to the PostgreSQL database. The system does the initial preprocessing of every email after which the AI module handles it.

As seen in Figure 1.2 the AI Processing and Triage Layer processes the support request using ML and NLP techniques. A fine-tuned DistilBert model is used to determine the intent of the customer's request and the priority level of the request. Information extraction using spaCy is used to determine the entities and details within the email message. The intent prediction is used to determine the appropriate destination of the support request, while the priority level is used in managing tickets.

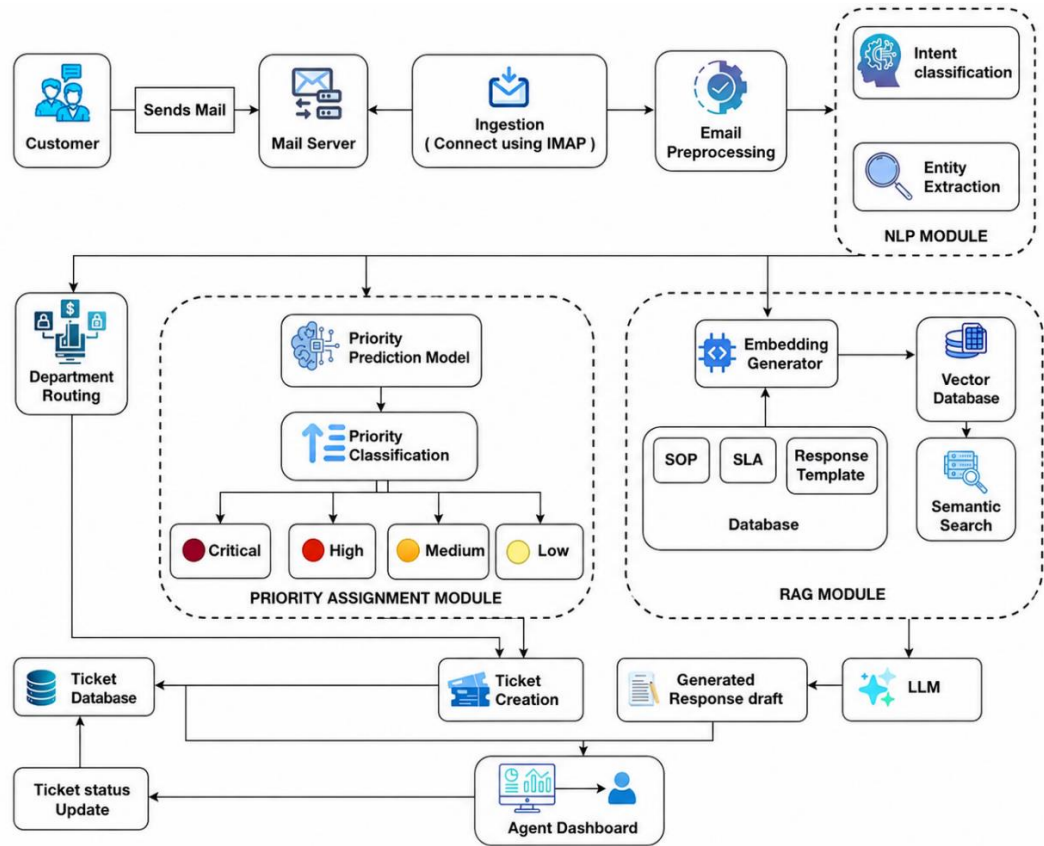


Figure 1.2. Overview of the proposed System Architecture

The Intelligent Routing Module determines the supported intents set up for the existing mailboxes and workspaces. If the mailbox being used is able to cater to the supported intent, the ticket stays within the same workspace. However, if the mailbox cannot process the intent, another eligible destination is found and the ticket routed to that destination. This process is documented for purposes of traceability during the lifetime of the ticket.

CHAPTER 2

LITERATURE SURVEY

This section of the paper provides an all-round review about multiple existing systems that holds similar objective and problem statements as that of the proposed project. The study of the existing papers will produce valuable insights for the construction of the proposed system.

2.1 LITERATURE REVIEW

Bonechi et al. researched the application of artificial intelligence to classify customer support requests in the banking domain. The researchers presented a full pipeline for ticket classification employing a real-life dataset of 4,243 customer requests split into ten categories. BERT-based classification was compared to the traditional TF-IDF and SVM classifier, proving the utility of NLP approaches to enhance the organization of customer support tickets. This work emphasizes the necessity of ticket classification as the first step in providing faster and more effective customer support.

Truss and Böhm explored customer support ticket classification by employing AutoML approaches. The research aimed at minimizing the effort of developing machine learning solutions by making models trainable without substantial specialization in machine learning. The research proved that AutoML can produce good classification results and can facilitate automatic ticket processing. The research mostly concerns classification and does not deal with the entire process of intelligent

routing, workload-based agent allocation, knowledge extraction.

Ahmed et al. conducted research on machine learning techniques for predicting the severity of IT incidents. It highlights the necessity to predict the severity level of the issue because the delay in resolving the matter can seriously influence the operations of the company. The paper contains an empirical analysis of classification techniques used for predicting the severity of incidents, which is evidence of the significance of machine learning for sorting support requests. Thus, the introduction of automated priority prediction is justified because urgent requests should be spotted prior to entering the regular support queue.

Xu et al. analyzed Retrieval-Augmented Generation in terms of customer service question answering using knowledge graph in the retrieval process. It identifies a drawback in conventional RAG techniques when the support information might be treated as a piece of unstructured text regardless of relations between the problem and related information. It shows how the use of structured knowledge can positively influence the retrieval process and generation of appropriate answers. The work justifies the introduction of knowledge retrieval in SmartAssist where corporate documents and previous support information can be used as context.

Nowak-Brzezińska and Drabek introduced an innovative technique based on the combination of the rule-based and machine learning techniques for the purpose of support ticket classification. According to the researchers, in the case of tickets that can be detected with high confidence, the rule-based approach is used, whereas in those cases when predefined rules are not enough, the machine learning technique is applied. As a result, the authors state that the hybrid approach

provides for reliable classification while maintaining transparency of the decision-making process based on the use of rules.

Jain et al. introduced an innovative AI-based approach to ticket management that helps automate the support process. In particular, various AI-based techniques such as ticket clustering, ticket prioritization, and ticket resolution recommendation are combined in an innovative architecture along with the enterprise tools. The major goals of this innovation are to decrease the amount of effort required in the processes involved, increase compliance with SLAs, and help to improve overall performance. Yet, the general architecture of the proposed solution leaves room for the development of email-based platforms.

In a separate study, Licina et al. designed HelpAgent, which is an explainable helpdesk system that utilizes automated ticket classification and human intervention alongside an explainable AI system. The framework allows users to accept or reject the proposed classification, which can then be explained by using large language models to simplify SHAP-based explanation. According to their results, the performance of the models was excellent, and the system was preferred by users. This evidence proves that the automation should not replace the human component entirely, which justifies the use of the Human-in-the-Loop approach in SmartAssist for request review, escalation, and resolution.

Carroll et al. tested the effectiveness of using large language models for support-ticket triage and response generation in a REDCap support environment. In particular, the researchers compared LLMs with and without organization-specific RAG resources for categorizing tickets and creating responses. The results show a high level of agreement in

ticket categorization, but the additional access to organizational documentation improves the response quality in the case of low and moderate complexity tickets. These results prove the practical relevance of using LLMs with domain-specific knowledge.

The research by Zangari et al. reviewed the literature on ticket automation and analyzed its applicability to the multi-level classification problem. The paper recognizes that the steps of tickets' categorization and routing are essential components of a more efficient support request processing time reduction. It also highlights the advantages of contextual language models and hierarchical label information in the classification. Thus, the present review proves the existence of ticket automation beyond classification.

Razma and Jurkus explore the usage of AI-powered classification of IT support requests in service-management environments in enterprises. The paper describes the use of artificial intelligence in the process of the classification of the incoming requests to enhance IT support management processes in the enterprise environment. It shows the increasing application of artificial intelligence to manage enterprise support request workloads.

The work of Dredze et al. laid down the foundation of intelligent email assistance when applied to AI-based email processing. Specifically, they noticed the problem of information overload due to increasing email traffic in the workplace environment and attempted to leverage artificial intelligence in order to address this issue. Even though the research was conducted before the advent of modern transformer and generative AI technologies, it provides a solid ground for the motivation behind

automatic email tasking.

The topic of automated email triaging in conjunction with SLA management has been tackled by Amalraj Victoire and Nithyalakshmi. They emphasized the problems associated with increasing volume of support emails and how the manual triaging process can take a lot of time. The automation-based solution shows how AI can be used to find and process the incoming requests in time. This area is strongly related to SmartAssist since the latter performs triaging, prioritization, routing, knowledge-assisted response generation.

2.2 COMPARISON AND DISCUSSION

When compared to the above discussed research papers and existing platforms like Zendesk and ServiceNow, the SmartAssist solution tackles various shortcomings through a more specialized and flexible approach. Most of the existing literature revolves around only one particular function, whether it is ticket classification, severity prediction, routing, or generating a response. In contrast to that, SmartAssist integrates all of them within an email support workflow. As opposed to other approaches relying mostly on the routing system or separate AI services, SmartAssist implements intent-based workspace routing based on supported intents in each mailbox and load-balanced agent assignment by active ticket load. It also uses RAG with pgvector for extracting appropriate SOPs, SLAs, FAQs, policies, and past resolutions prior to LLaMA providing a response. Human-in-the-loop reviews and corresponding escalation and confidence-based strategies add flexibility and reliability to the solution.

2.3 CONCLUSION

From the literature survey, it is evident that AI has made huge improvements in the automation of customer support through ticket classification, prioritization prediction, routing, knowledge retrieval, and auto-response generation. Nevertheless, the literature survey suggests that there is a significant gap in the study of an all-in-one email support automation framework because most of the available studies deal with either one process or involve a complicated platform configuration. Additionally, the available literature points out the necessity of precise classification, specific domain knowledge, explainability, and human supervision. Hence, considering the above findings, SmartAssist will address the noted gap by integrating intent and priority prediction along with intent-based routing of workspace, work load-balanced agent assignment, knowledge retrieval using pgvector and RAG, and LLaMA-based response generation with human supervision.

CHAPTER 3

SYSTEM DESIGN

This chapter of the paper briefly discusses about the design of the proposed system with the help of model diagrams. The requirements of the system and deployment are also extensively studies. This chapter also covers the overall architecture of the proposes system along with overview of modules involved.

3.1 SYSTEM ARCHITECTURE

The Email Integration module acts as an initial step, integrating organizational support mailboxes via different services, like Gmail API, IMAP or Microsoft Graph. The module takes the incoming emails and their metadata information (sender, recipient, subject, message body, etc.). A Choose Mailbox module allows the user to choose the connected mailbox from which emails will be processed. The received emails are being further passed to the Preprocessing module, where the information is cleaned and normalized, removing extra formatting and handling missing or unnecessary characters, readying the text for further AI processing. This way the consistent input information is guaranteed to be provided for the NLP models for further predictions.

AI Processing and Triage is where the main analysis of the incoming support emails happens. Intent classification model is built using DistilBERT and predicts the kind of request that comes in the email, and the Priority prediction also uses DistilBERT model in order to estimate the urgency of the ticket. Also, the architecture involves

Information extraction based on spaCy library. On the basis of predicted intent and intents which are supported for connected mailboxes, the Department/Workspace Routing This approach separates prediction from routing, allowing the system to use AI for understanding the request while applying configurable organizational rules for assigning it to the correct support workspace.

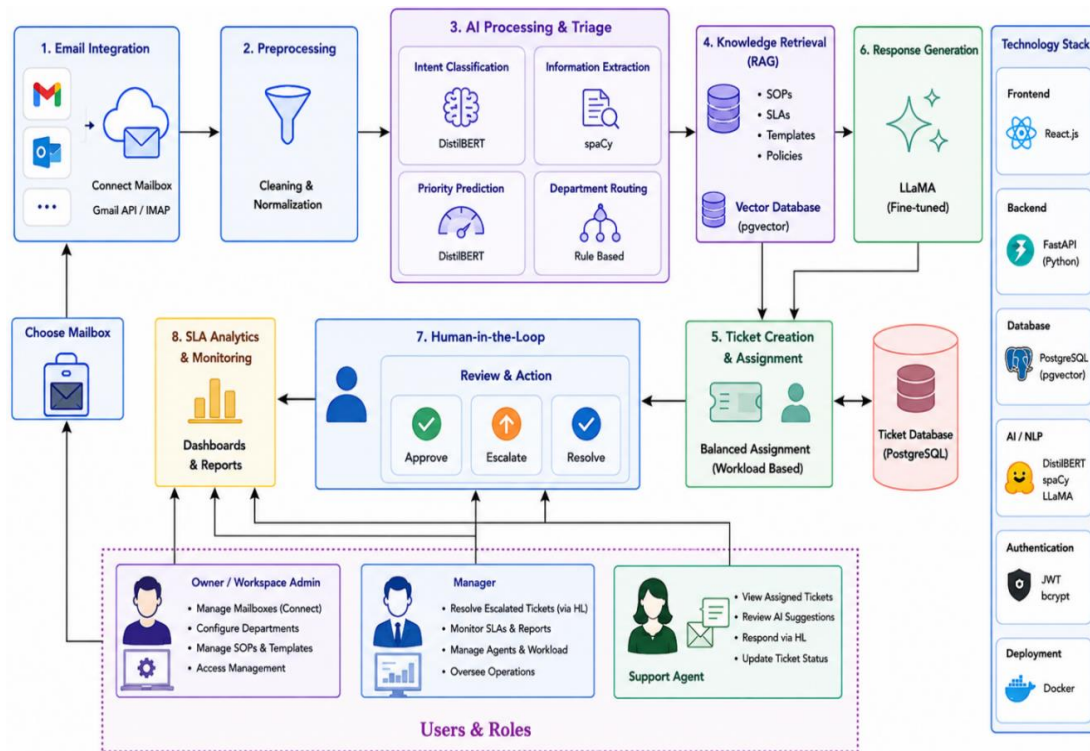


Figure 3.1. Architecture diagram for the entire system

Once the email has been processed, the knowledge retrieval (RAG) module extracts relevant information from the company's knowledge base. The documents like SOPs, SLAs, templates, and policies are converted into embeddings and then stored in the PostgreSQL database via the pgvector. Relevant information is extracted based on the content of the ticket and is used as contextual information for response

generation. The Response Generation module creates a context-aware response to the email using LLaMA-based model. In this way, responses can be generated using the company's knowledge instead of using the general knowledge of the model while being consistent with the company procedures and policies.

After the email is analyzed, the ticket creation and assignment module converts it into the ticket which contains information about the customer, the subject of the email, its description, predicted intent, priority, status, and generated response. After determining the destination workspace, tickets are assigned to the available agents in a balanced way considering the number of open tickets of each agent. The Human-in-the-Loop module then provides the support team with control over the AI-generated results. Managers and support agents can review the ticket and AI suggestion, after which the ticket can be approved, escalated, resolved, or updated according to the situation. This ensures that automated processing remains subject to human supervision.

The architecture is designed to accommodate three types of users: Owner/Workspace Admin, Manager, and Support Agent. The Owner oversees the management of mailboxes, departments/workspaces, SOPs, templates, and access control. The Manager oversees operations, SLAs, reporting, workload, and escalated tickets. And the Support Agent reviews tickets, evaluates the AI suggestions, and replies to customers. The ticket data is stored in the Ticket Database, which is built on top of PostgreSQL. For SLA analytics and monitoring, the system has its own module. It provides dashboards and reports for monitoring support efficiency. The architecture is built with React.js (front-end), FastAPI (back-end with

Python), PostgreSQL with pgvector (structured and vector database), DistilBERT, spaCy, LLaMA (AI/NLP), JWT (authentication), and Docker.

3.2 SYSTEM REQUIREMENTS

This section describes the requirements of the proposed system. Various aspects of the system like the hardware and software compatability, the data requirements, dependencies and other critical requirements are discussed in detail.

3.2.1 SOFTWARE REQUIREMENTS

Python is used as the main programming language to develop the backend and AI components as it supports machine learning, NLP, API development, and database operations. JavaScript is used for frontend using React.js to provide an interactive user interface to manage workspace, tickets, get AI suggestions, and analytics.

FastAPI is used to develop the backend of SmartAssist that offers REST APIs for performing tasks related to email processing, ticket handling, user authentication, routing, and AI services. For AI components, PyTorch, Transformers, spaCy, and Sentence Transformers are used to train models and perform tasks like intent prediction, priority prediction, information extraction, and text embedding respectively. The RAG component works with pgvector and PostgreSQL to store and retrieve the vector embeddings of the knowledge of the organization including SOPs, SLAs, policies, templates, and FAQs.

Table 3.1. Software requirements of the proposed system

Software Component	Suggested Version (or) Release	Purpose
Python	3.14	Primary language for backend ,NLP and automation
React.js	19.2.7	Develops the interactive user dashboards.
FastAPI	0.141.1	Provides REST APIs for system and AI services.
PyTorch	2.14.0	Supports deep-learning model training and inference.
Hugging Face Transformers	5.16.0	Implements and fine-tunes DistilBERT models.
spaCy	3.8	Extracts relevant information and entities from emails.
PostgreSQL	17	Stores users, emails, tickets, workspaces, and system data.
pgvector	0.7+	Enables vector storage and similarity search for RAG.
LLaMa	Compatible selected release	Generates context-aware responses using retrieved knowledge.
Visual Studio	Latest stable	Development, debugging, dependency management, and

		project execution.
--	--	--------------------

For development, Visual Studio Code can act as the main Integrated Development Environment, along with a Python virtual environment, which will be used to ensure that there is no conflict in the backend dependencies. Node.js and npm have to be installed in order to install React frontend packages. Windows 10 or Windows 11 can be used as the system, while Linux systems can also be used to deploy the solution. Git is used to control the source code, while Docker can be used to deploy everything.

3.2.2 HARDWARE REQUIREMENTS

SmartAssist needs a computer having enough computing power and memory to execute the backend services, PostgreSQL database, frontend application, NLP models, and the RAG pipeline at once. The use of a multi-core CPU is advised in dealing with simultaneous API calls, emails, database queries, and inference from models. 8 GB RAM will suffice for development and normal use, but 16 GB or higher will be good in case of using the AI models and database locally.

In case of model training or fine-tuning of DistilBERT and LLaMA, especially, a GPU will speed up the process. But GPU is not essential in the case of normal use of the app if the models have been trained beforehand. SSD is also needed for storing the operating system, source code, dataset, PostgreSQL database, models, and RAG knowledge-base embeddings.

Table 3.2 Hardware Requirements of the proposed system

Hardware Component	Suggested Specification	Purpose
Processor (CPU)	Intel Core i5 / AMD Ryzen 5 or above	Runs backend, frontend, database, and AI inference tasks.
RAM	16 GB	Supports simultaneous execution of application services and AI models.
GPU	NVIDIA GPU with 6-8 GB VRAM or above	Accelerates model training and AI inference when GPU processing is required.
Storage	512 GB SSD	Stores datasets, models, source code, database, and system files.
Server	Windows or Linus Server with multiple processing cores	For the provision of suitable development environment
Network	Stable broadband or Wi-Fi connection	Required for email API integration, external services, and deployment.

3.2.3 DATASET REQUIREMENTS

The system needs Email, Classification, Knowledge base, and Response datasets to train the AI models as well as to develop the RAG and response generation modules.

1. Email Dataset – Email subject, content, from, to, and timestamp of support emails.
2. Intent Dataset – Support emails categorized according to the respective intent/category.
3. Priority Dataset – Support emails labeled as Low, Medium, High, or Critical priority.
4. Knowledge Base – SOPs, SLAs, policies, templates, and troubleshooting guides for RAG.
5. Response Dataset – Historical approved support responses used as reference for generating responses.

3.2.4 DEPLOYMENT AND SCALING

The entire proposed system is maintained in the server, which can be an On-premise server, or a cloud server. The server is to be maintained continually, and must be highly scalable to accommodate with a minimum of 2TB of memory and a 16GB processing RAM to handle the online customer support emails sent.

3.3 SYSTEM DESIGN

This section deals with the properties of UML diagrams to plan the development process of the overall system and also to provide a clear picture of the system workflow and communication between the components.

3.3.1 ACTIVITY DIAGRAM

The activity diagram for the active environment includes the sequence of activities carried out by the SmartAssist when providing the email-support service. The diagram indicates how the inbound support email is ingested, preprocessed, analyzed based on the AI algorithm for intents and priority determination, information is extracted from the email, the knowledge base is retrieved using the RAG approach, the reply message is generated, recommended, reviewed by humans, actions to be taken are approved, etc.

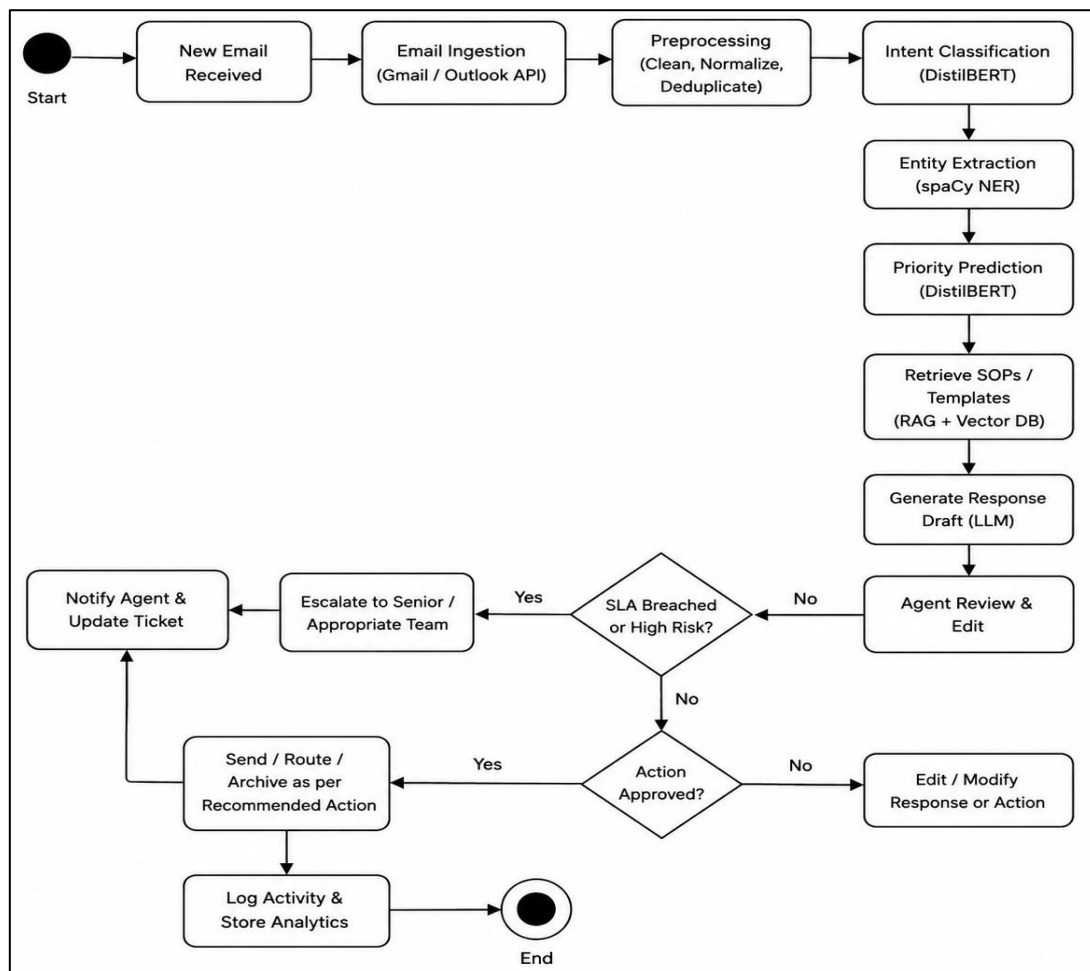


Figure 3.2. Activity Diagram for the Active Environment

The activity diagram illustrated in Figure 3.2 is created as a flow-based sequence starting from receiving the support email and ending up with logging and analyzing the activities. The SLA breach or high risk detection and its handling are shown as well as other decision. In this diagram, rounded rectangles denote system activities, diamonds denote decisions, and arrows indicate the flow of the process from one node to another.

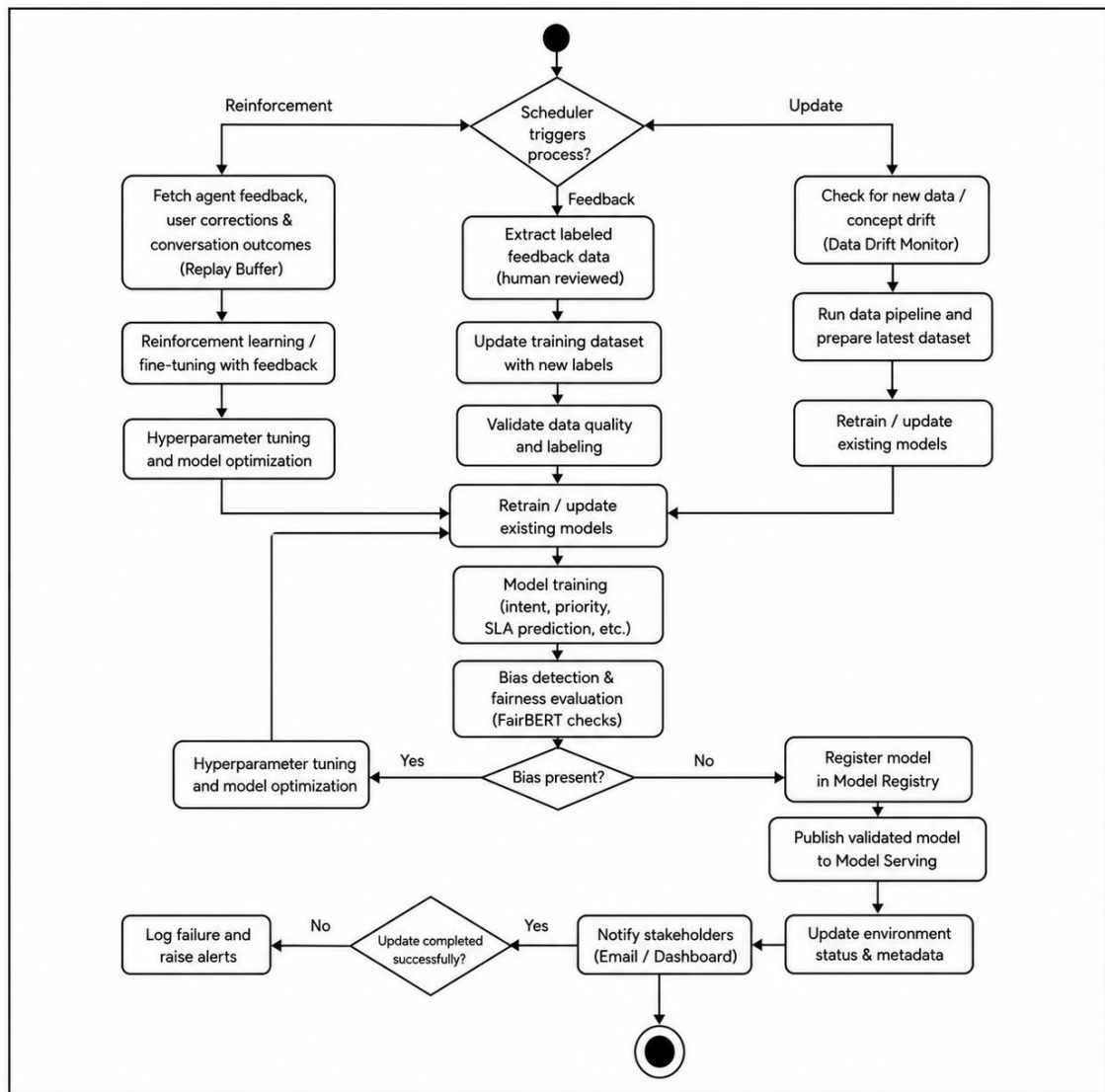


Figure 3.3. Activity Diagram for the Passive Environment

Activity Diagram depicting passive environment in the Figure 3.3 captures the process of background learning and enhancement of the SmartAssist system. The diagram is designed with two primary flows that are caused by the scheduler: reinforcement learning with feedback and data update with drifted data. These processes are merged together in order to perform retraining, validation, bias checks, and deployment. Lastly, the environment metadata is updated and stakeholders notified.

3.3.2 DATAFLOW DIAGRAM

Data Flow Diagram (DFD) is the pictorial depiction of how the data flows within the system. It represents where data comes from, what processes are applied, where it is stored and how it flows through all these steps. Though DFD is not a common UML diagram, it is widely used together with other UML diagrams in analyzing and designing systems as the illustration of data movement and processing.

In case of SmartAssist, the process starts with the email sent by the Support Agent that comes through the Email Ingestion module. Then the raw email goes through Preprocessing to be normalized and cleaned and then goes through NLP Processing in order to identify intent, entities and priority of the message. NLP module communicates with the Knowledge Base in order to get the proper SOPs and templates.

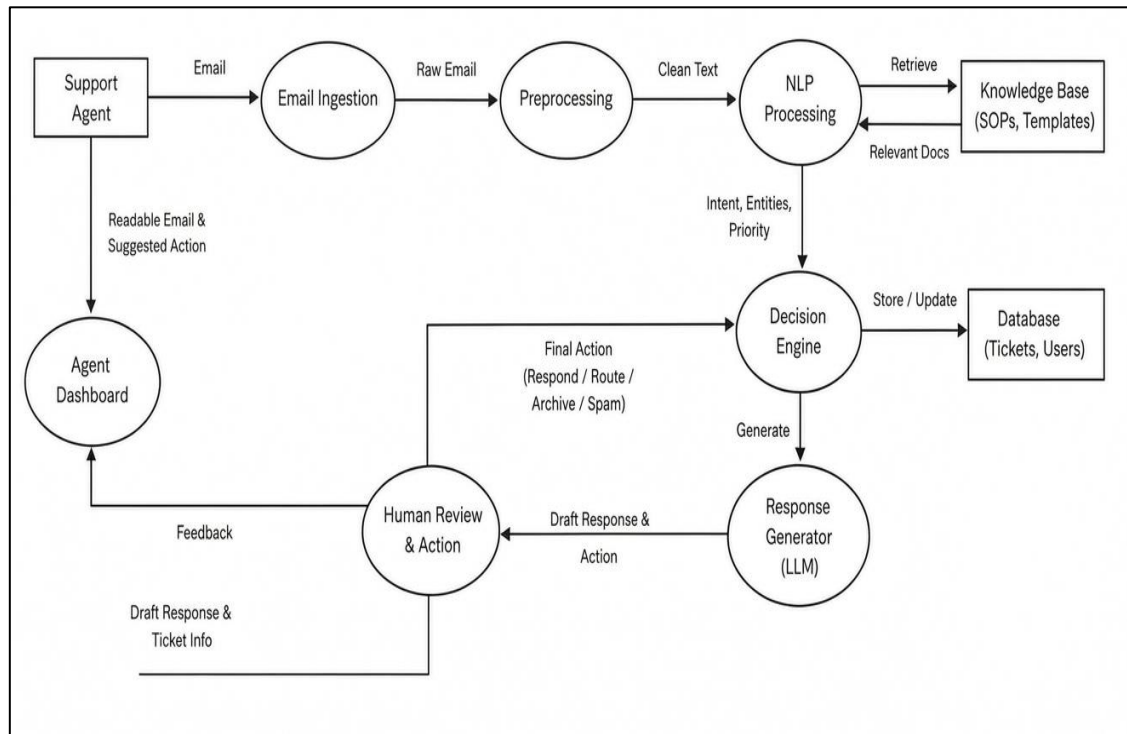


Figure 3.4. Data Flow Diagram of the system

The Figure3.4 demonstrates how the processed data is sent to Decision Engine, which updates the database with ticket and user information and generates the response using the LLM. Then, the generated draft and suggested actions go through Human Review & Actions, where the agent may check and give feedback. The approved action and the ticket information is shown in the Agent Dashboard while the feedback is being sent back into the system for continuous improvement of the performance of the entire proposed system.

CHAPTER 4

PROJECT DESCRIPTION

4.1 METHODOLOGIES

The SmartAssist approach consists of a multi-stage AI-driven process of processing support emails and transforming them into tickets. First, emails are gathered and processed by cleaning and normalization. Intent classification and priority prediction based on DistilBERT are executed, and relevant information is extracted with spaCy. Depending on the predicted intent rule-based workspace routing is done, followed by the assignment of the ticket to an appropriate support agent using workload balancing. RAG with pgvector is used to fetch the relevant organizational knowledge like SOPs, SLAs and templates. This information is then passed on to the LLaMA-based response-generation model for the creation of the response. Human-in-the-loop approach is employed in order for the agents/managers to be able to revise, edit, approve, escalate, or close the ticket prior to taking the ultimate action.

4.2 MODULES

The SmartAssist system is divided into five major modules covering the complete machine-learning workflow, from dataset collection to final model evaluation. Each module performs a specific function and contributes to the development of an effective email triage and response automation system.

4.2.1 DATASET COLLECTION AND PREPROCESSING

An open-source IT customer support email dataset from Kaggle has been employed and modified for SmartAssist. The final dataset consists of 5,000 emails having attributes of content, body, intent, and priority. Content and body have been merged to keep the request summary as well as its context intact. The dataset has been preprocessed to eliminate missing data, duplicates, HTML, unimportant formatting, and unnecessary metadata. It has been modified to fit into 25 intent categories and 3 priority categories (High, Medium, Low). Stratification was applied when splitting to maintain the distribution of classes and resulted in 3,500 training, 750 validation, and 750 testing emails. Texts have been tokenized using distilbert-base-uncased tokenizer with max_length of 256.

Table 4.1 All the 25 Intents used for model training

Intent	Intent	Intent
Account Access	Password Reset	Access Permission
Account Locked	Payment Failed	VPN Issue
Application Error	Refund Request	Network Connectivity
Billing Issue	Security Incident	Email Issues
Complaint	Service Outrage	Slow Performance
Duplicate Payment	Software Bug	Data Export Request
Feature Request	Subscription Issue	Two Factor Authentication
General Support	Invoice Issue	Password Issue
Login Issue	-	-

Table 4.2 Distribution of dataset across training and testing sets

Dataset Split	Percentage
Base Model	DistilBERT (distilbert-base-uncased)
Architecture	Multi-task DistilBERT
Intent classes	25
Priority classes	3
Training Samples	985
Validation Samples	211
Test Samples	212
Epochs	5
Batch Size	16
Learning Rate	2×10^{-5}
Weight Decay	0.01
Dropout	0.2
Maximum Sequence Length	256 tokens
Optimizer/training Framework	Hugging Face Transformers / PyTorch
Hardware	NVIDIA Tesla T4 GPU

4.2.2 MODEL TRAINING AND EVALUATION

A multi-task DistilBERT model was trained over 5 epochs with 25 classes of intents and 3 classes of priorities using dropout (0.2), batch size 16 and weight decay 0.01. Training was conducted with FP16

on a NVIDIA Tesla T4 GPU. Loss values during training and validation periods were decreasing regularly, hence indicating successful convergence. This model showed 88% intent accuracy and 82% priority accuracy on the test dataset with confidence of 86.34% and 95.15%, respectively.

4.2.3 EMAIL INTEGRATION AND TICKET GENERATION

The E-mail Integration module links SmartAssist to organization's mailboxes through Gmail API, Microsoft Graph API or IMAP. E-mails, along with their metadata, are fetched and are sent for further processing. Post processing, DistilBERT determines the intent and priority of e-mail, while spaCy extracts significant information from the email message. An automatic creation of tickets takes place that contains information like E-mail subject, E-mail description, customer information, intent, priority, status and mailbox.

4.2.4 DEPARTMENT ROUTING AND AGENT ASSIGNMENT

After calculating the predicted intent, priority, and confidence, the routing service selects the workspace or department based on the configured department-intent mappings. It checks the predicted intent against the supported intents of the available departments to determine the proper destination. In case the confidence is less than the threshold value or there is no appropriate mapping, the ticket is routed to the general manual review queue. Once the department is determined, the ticket is assigned to an eligible support agent following a workload-based algorithm. Each agent's workload is computed considering the tickets that

are currently handled by him and their priorities as follows:

$$W_i = O_i + \alpha H_i + \beta C_i$$

where W_i represents the workload of agent i , while O_i , H_i , and C_i denote the agent's open, high-priority, and critical-priority tickets, respectively. The agent with the minimum workload is selected:

$$\text{AgentSelected} = \text{Minimum}(W_i)$$

In case two or more agents have equal workload, the senior agent with a longer tenure in the workspace is chosen as a tie-breaker. The allocation is done inside a database transaction to ensure consistency while allocating tickets in batch mode. This component facilitates cooperation between the predictions made by the AI model and organizational routing rules.

4.2.5 KNOWLEDGE RETREIVAL AND RESPONSE GENERATION

In this module, RAG is used to retrieve knowledge, manage SLA and generate responses with LLaMA. There is a separate knowledge base for each workspace which includes SOPs, response templates, FAQs, and policies. After uploading a document, we extract information from the documents, do segmentation, generate Sentence Transformer embeddings and store them in PostgreSQL with pgvector. Then, when a ticket arrives, we compare its embedding to the stored document vectors and retrieve relevant knowledge chunks. At the same time, we find the SLA policy that applies to this ticket depending on the ticket's priority, calculate deadlines and monitor them constantly. The SLA status is ON TRACK, AT RISK, BREACHED, or MET, and notifications are sent whenever needed.

Finally, we pass the original email, extracted information, intent, retrieved SOPs, SLA information and approved templates as context to the LLaMA response-generation model which generates an agent-ready response avoiding unsupported policies, dates, actions, and commitments.

4.2.5 HUMAN-IN-THE-LOOP APPROVAL

Human-in-the-Loop acts as the last module that allows for controlling of the work done by AI tools. Either the assigned support agent or manager will check the ticket, predicted data, knowledge acquired, and the response proposed by the AI tool. It is possible for the user to either approve the proposed action, modify it, escalate, resolve, reroute, or even reject it. Any approved responses will be forwarded to the client, while any changes made will be logged into the system for future reference.

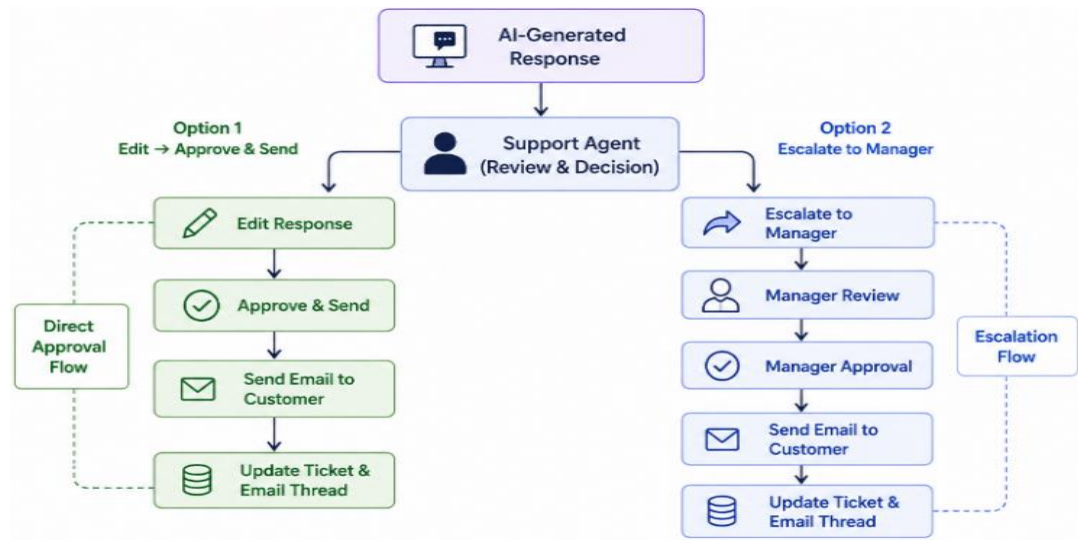


Figure 4.1 Review and Response by Support Agent

CHAPTER 5

IMPLEMENTATION AND RESULT DISCUSSION

5.1 IMPLEMENTATION AND RESULT

The SmartAssist project was designed as a web application for an intelligent email support system that combined ingestion of emails, AI-powered classification, information extraction, routing, knowledge retrieval, response generation, and human oversight. The backend was written using Python and FastAPI, whereas the frontend was created with React.js. For storing structured data, PostgreSQL was chosen, and pgvector was used for implementing vector search.

5.1.1 USER INTERFACE AND EMAIL INTEGRATION

With the React-based interface, the users have role-based access as Owner, Manager, or Support Agent. The dashboard shows various statistics, priorities, statuses, assignments, and the latest support activity. The support email accounts can be linked using Gmail API, Microsoft Graph API, or IMAP. The incoming emails are fetched along with the subject, body, sender, recipient, and time. The connected mail box is continuously monitored for any new arriving mails with a time period of 5 seconds. The dashboard for login and support agent workspace is illustrated in Figure 5.1 and Figure 5.2.

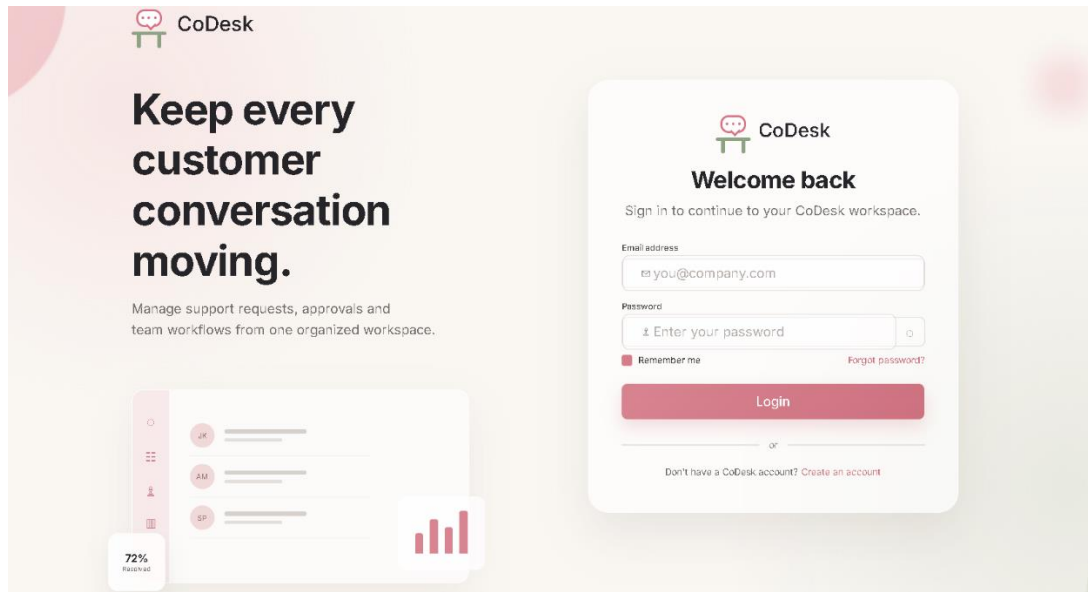


Figure 5.1 SmartAssist Login Interface

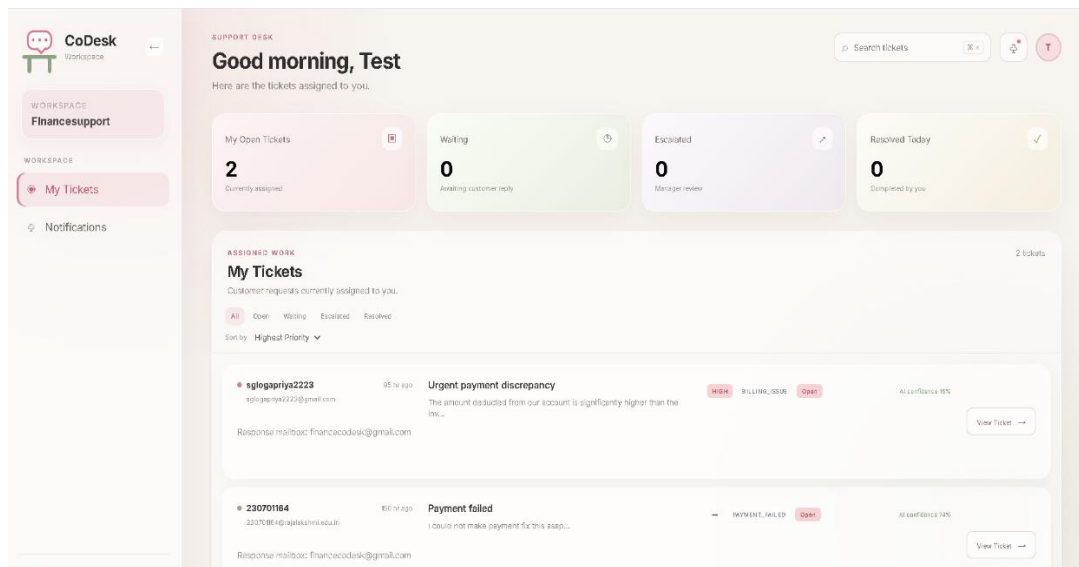


Figure 5.2 SmartAssist Support Agent's Dashboard Interface

Once this is done, the email gets cleaned and normalized. The subject and body of the email get combined and tokenized using the distilbert-base-uncased tokenizer with the maximum token limit of 256.

5.1.2 MODEL PERFORMANCE EVALUATION

Multi-task DistilBERT model was used to carry out the task of intention classification and priority prediction. The model was trained on a dataset of 5,000 IT support emails which had 25 intents classes and three priority classes, including High, Medium, and Low priorities. The dataset comprised 3,500 training samples, 750 validation samples, and 750 test samples.

Table 5.1 Classification Model Evalutation

Evaluation	Intent	Priority
Validation Accuracy	85%	89%
Validation Precision	84%	81%
Validation Recall	86%	79%
Validation F1	78%	82%
Validation Average Confidence	86.06%	95.07%
Test Accuracy	88%	82%
Test Average Confidence	86.34%	95.15%
Test Minimum Confidence	72.37%	88.78%
Test Predictions $\geq 70\%$	660/750	615/750

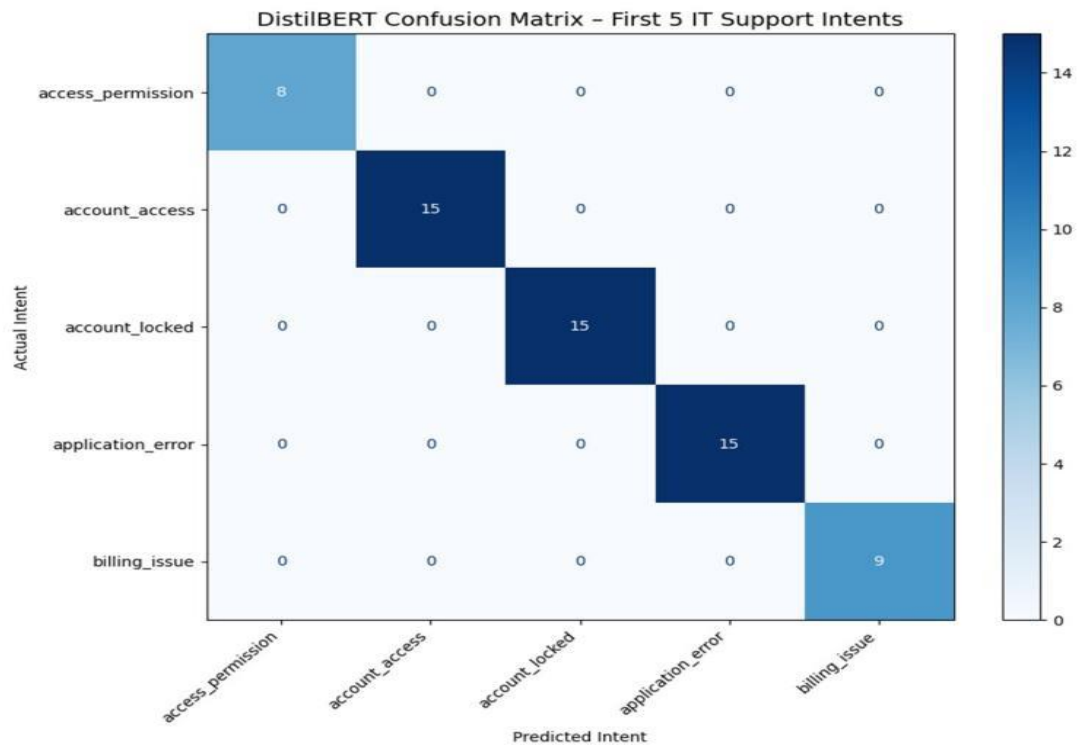


Figure 5.3 Confusion Matrix for the support intents

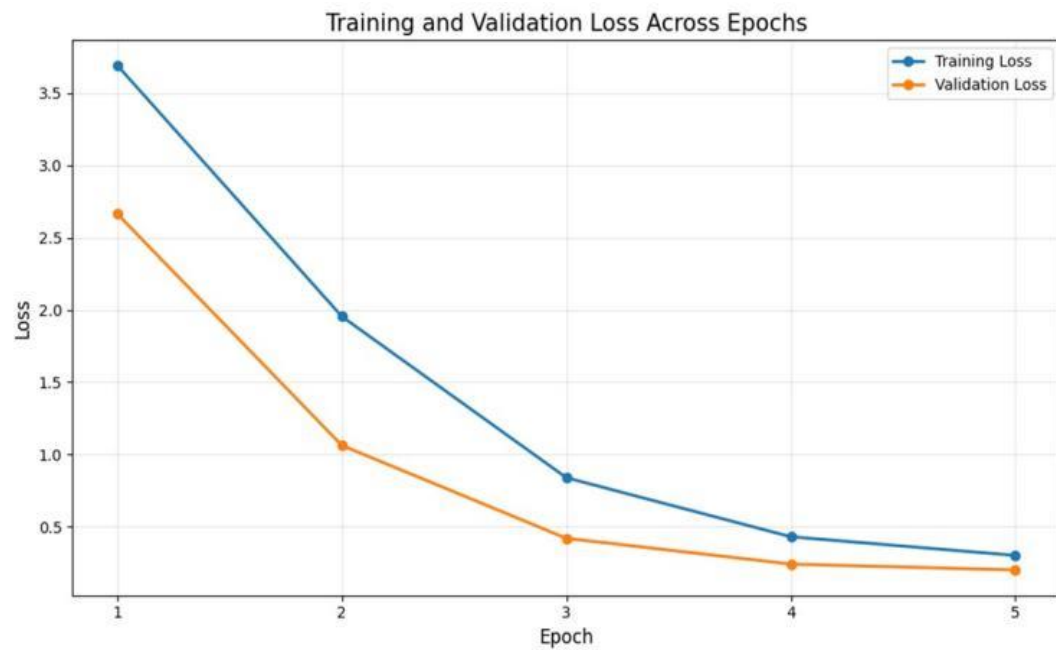


Figure 5.4 Training and Validation Loss Across Epochs

Model training was carried out for five epochs with the batch size 16, learning rate 2×10^{-5} , dropout 0.2, and weight decay 0.01. The model was trained via FP16 training on NVIDIA Tesla T4 GPU. SpaCy was employed to extract relevant information from the email.

5.1.3 ROUTING AND RESPONSE GENERATION

The intent which is predicted is matched with the supported intents that are enabled for the organizational mailboxes. In case the current mailbox supports the intent, the ticket stays in the current workspace; otherwise, an appropriate destination is selected. Unclear tickets are referred for manual routing. Then the ticket is handed over to an available support agent considering the active workloads.

In order to generate the response, the organization's documents including SOPs, policies, SLAs, and approved templates are split into chunks and embedded using Sentence Transformer. These embeddings are saved in PostgreSQL via pgvector. The relevant knowledge is retrieved using similarity search and is provided to the LLaMA-based response generator as context. The agent or manager may accept, modify, escalate, resolve or reject the suggested response prior to sending it to the customer.

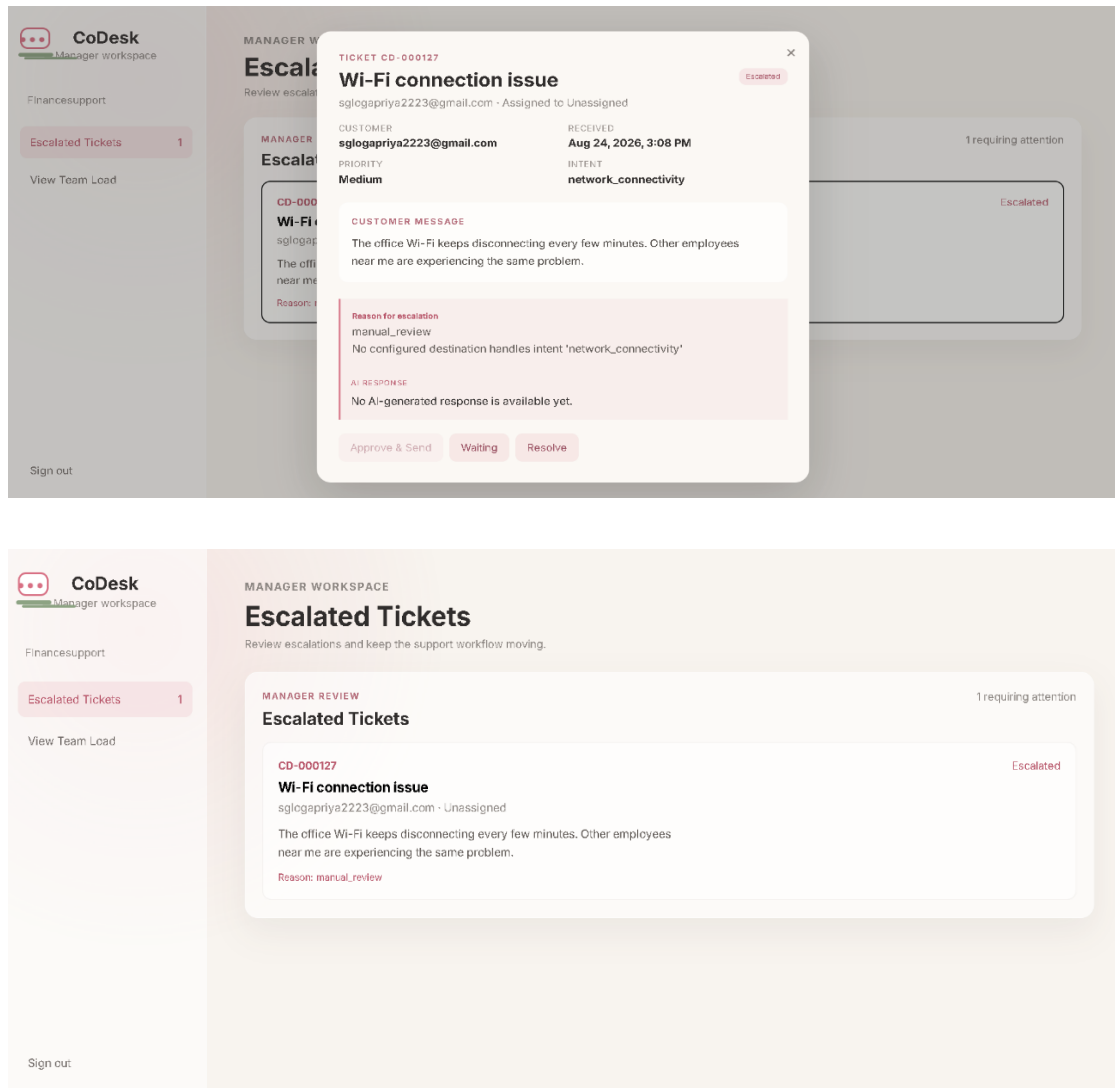


Figure 5.5 Ticket Review and Human-in-the-Loop Interface

5.2 DISCUSSION

The obtained results prove that SmartAssist can automate successfully several key steps of the email-based customer support process. The 88% accuracy of intent classification implies that the model is capable of classifying correctly the vast majority of support tickets in 25 different classes. The prediction of priority demonstrated 82% accuracy, but its

higher average confidence (95.15%) speaks about higher confidence in predictions for three broader classes of priority. Training and validation loss values demonstrate that the model was trained stably; however, lower F1-score of intent classification compared with the accuracy value can speak about difficulties of distinguishing some of the classes. Combining RAG and LLaMA expands the functionality of the system through enabling the generation of responses using the knowledge specific to each organization. Human-in-the-loop allows controlling this process since the support personnel can confirm the answer generated by AI. Thus, SmartAssist includes email integration, triage with the help of AI, routing based on intent identification, assignment based on workload, knowledge retrieval using RAG and LLaMA and human-in-the-loop among others.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The proposed intelligent email triaging and response automation system provides an integrated approach to automating the key functions of customer support and IT service management. The proposed system involves a transformer-based classification and information extraction from emails, deterministic routing, allocation of agents depending on the workload, workspace isolated retrieval augmented generation, SLA management, and Llama-based response generation process. The segregation of the prediction process based on AI from rule-based routing, along with human intervention before any contact with a customer, allows not only to increase the level of automation but also to provide proper control and security. The workspace isolation combined with role-based access allows providing multi-tenancy, and the continuous email monitoring, instant notifications, and SLA tracking allow managing incoming emails properly. The usage of the retrieval-augmented generation mechanism helps ground the answers to organization-specific SOPs and policies. Thus, the proposed solution is expected to provide improved automation of the triage process, increased consistency of classification and ticket assignments, support-agent workload balancing, and reduced customer response time while keeping the necessary level of control. mechanism, and provide additional features like automated escalation and analytics of the support emails.

6.3 FUTURE WORK

For future work, the goal will be testing SmartAssist with a bigger and more varied dataset of support emails to increase generalizability of the models. The classification model can be optimized through inclusion of more intents and continuous updates of training data. Another potential area for improvement is to enable the system to work with multiple mailboxes and email clients by enabling ingesting and tracking of emails across all organizational email addresses. Support across multiple languages and domain-specific entity extraction are other possibilities. In addition, the retrieval and re-ranking in the RAG pipeline can be optimized. Some of the features that could be included in the future are SLA prediction, escalation identification, and workload forecasting.

REFERENCES

1. S. Bonechi, G. Palma, M. Caronna, M. Ugolini, A. Massaro, and A. Rizzo, “Enhancing Customer Support in Banking: Leveraging AI for Efficient Ticket Classification,” *Procedia Computer Science*, vol. 246, pp. 128–137, 2024, doi: 10.1016/j.procs.2024.09.235.
2. M. Truss and S. Böhm, “AI-based Classification of Customer Support Tickets: State of the Art and Implementation with AutoML,” *arXiv preprint arXiv:2406.01789*, 2024, doi: 10.48550/arXiv.2406.01789.
3. S. Ahmed, M. Singh, B. Doherty, E. I. Ramlan, K. Harkin, M. Bucholc, and D. Coyle, “An Empirical Analysis of State-of-Art Classification Models in an IT Incident Severity Prediction Framework,” *Applied Sciences*, vol. 13, no. 6, Art. no. 3843, 2023, doi: 10.3390/app13063843.5
4. Z. Xu, M. J. Cruz, M. Guevara, T. Wang, M. Deshpande, X. Wang, and Z. Li, “Retrieval-Augmented Generation with Knowledge Graphs for Customer Service Question Answering,” *arXiv preprint arXiv:2404.17723*, 2024.
5. A. Nowak-Brzezińska and D. Drabek, “A hybrid knowledge-based machine learning approach for ticket classification in the AWS cloud,” *Procedia Computer Science*, vol. 270, pp. 4796–4803, 2025, doi: 10.1016/j.procs.2025.09.605.
6. S. Jain, A. Gupta, and K. Neha, “AI Enhanced Ticket Management System for optimized Support,” in *Proceedings of the 4th International Conference on AI-ML Systems (AIMLSystems ’24)*, 2024, Art. no. 21, pp. 1–7, doi: 10.1145/3703412.3703433.
7. E. Licina, A. Sartipi, Q. Zang, I. Tchappi, and A. Khadangi, “HelpAgent: An Explainable Framework for Transparent and User-Centered Helpdesk Ticket Classification,” *Procedia Computer Science*, vol. 280, pp. 654–662, 2026, doi: 10.1016/j.procs.2026.04.083.
8. A. Carroll, D. Brown, D. Silvio, J. Maszatics, and D. A. Hanauer, “Evaluating large language models for automated REDCap support ticket triage and response,” *JAMIA Open*, vol. 9, no. 3, Art. no. ooag073, 2026, doi: 10.1093/jamiaopen/ooag073.

9. A. Zangari, M. Marcuzzo, M. Schiavinato, A. Gasparetto, and A. Albarelli, "Ticket automation: An insight into current research with applications to multi-level classification scenarios," *Expert Systems with Applications*, vol. 225, Art. no. 119984, 2023, doi: 10.1016/j.eswa.2023.119984.
10. A. Razma and R. Jurkus, "AI-Based Classification of IT Support Requests in Enterprise Service Management Systems," *Systems*, vol. 14, no. 2, Art. no. 223, 2026, doi: 10.3390/systems14020223.
11. M. Dredze, H. M. Wallach, D. Puller, T. Brooks, J. Carroll, J. Magarick, J. Blitzer, and F. Pereira, "Intelligent Email: Aiding Users with AI," in *Proceedings of the 23rd AAAI Conference on Artificial Intelligence*, Chicago, IL, USA, 2008, pp. 1524–1527.
12. T. Amalraj Victoire and S. Nithyalakshmi, "AI-Driven Automation of Centralized Email Triage and SLA Management," *International Advanced Research Journal in Science, Engineering and Technology (IARJSET)*, vol. 13, no. 5, 2026, doi: 10.17148/IARJSET.2026.135103.

